

به نام خدا

گزارش تکمیلی آزمایش 5 – ضرب
کننده BOOTH – درس آزمایشگاه DSD

امیر محمد شربتی (402106112) – پوریا رحمانی (402111418)

1404/9/12

همانطور که در پیش گزارش هم ذکر شد، متأسفانه کدی که در پیش گزارش ارسال شد، مشکل داشت.

پس از کلی تلاش و تست و بازبینی و... خلاصه موفق شدیم کد را دیباگ کنیم. این گزارش شامل توضیحات کد اصلاح شده و همچنین کارهای صورت گرفته در آزمایشگاه است.

توضیحات تکمیلی و اصلاحات پیاده سازی

این پروژه شامل 5 ماژول است. یک ماژول تست و چهار ماژول برای پیاده سازی مدار:

- ماژول control_unit.v که مربوط به کنترل سیگنال های مدار است
- ماژول datapath.v که همان مسیر گذار داده است.
- ماژول booth_mult.v که ماژول اصلی و متصل کننده دو بخش قبل (datapath , controller) است.
- ماژول find_first_different_bit.v که جایگاه اولین بیت صفر و بیت 1 از راست را بدست می آورد.
- و ماژول booth_mult_tb.v که ماژول تست این مدار است.

طبق فرموده سوال باید datapath و control unit را جدا می کردیم و بعد آن دو را به هم وصل می کردیم که این کار را با booth_mult انجام دادیم. نکته مضاعفی که سوال می خواهد این است که بتوان در یک کلاک بیش از یک بیت شیفت را انجام داد تا سرعت مدار نسبت به ضرب معمولی بیشتر شود.

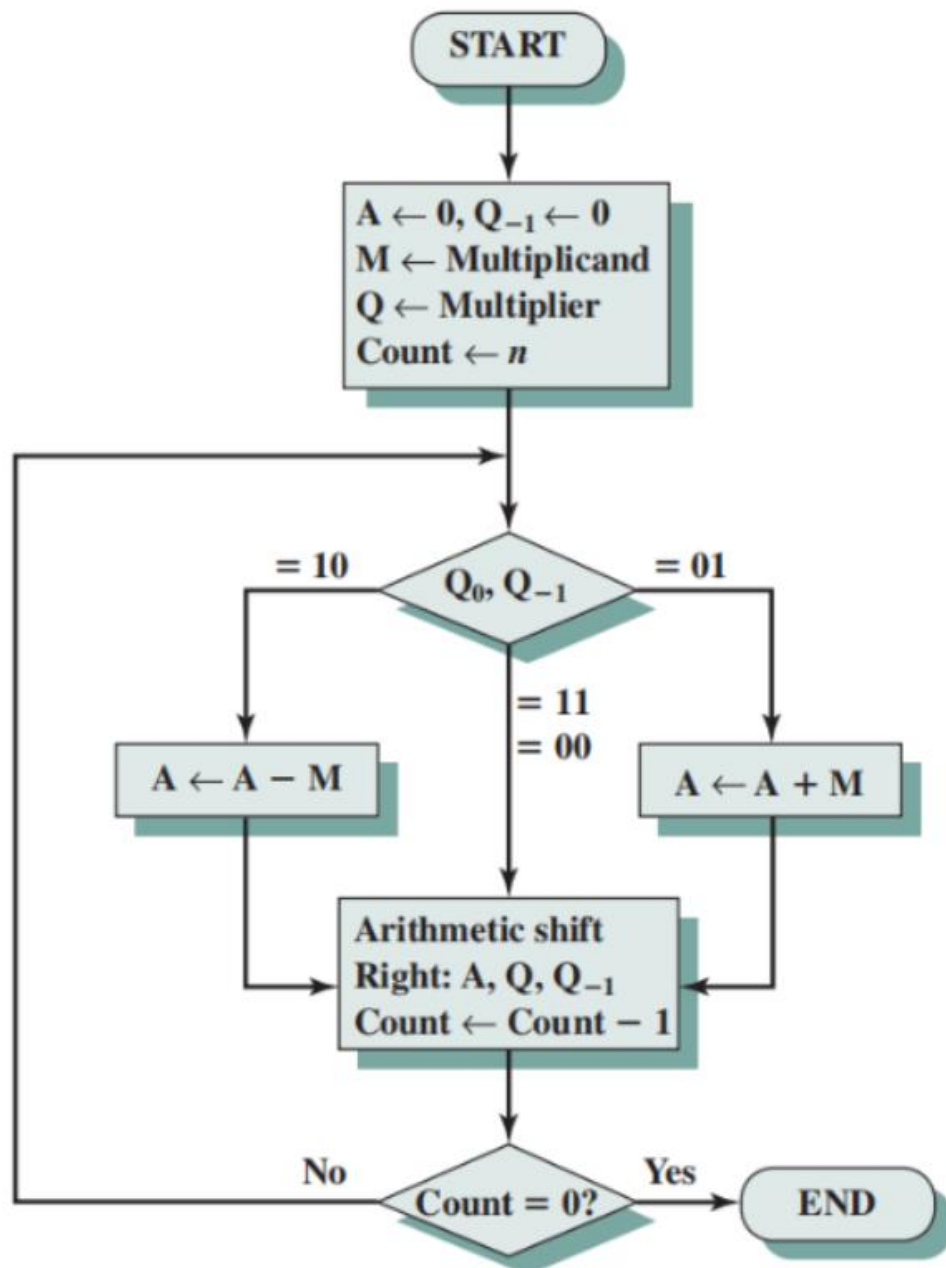
بنابراین ماژول find_first_different_bit.v پیاده سازی شد. این ماژول بر اساس پیمایش عدد از چپ به راست، ایندکس اولین بیت متفاوت از راست را حساب میکند. میتوان این منطق را با پیمایش از راست به چپ هم به راحتی انجام داد ولی این منطق ساده تر است.

در نهایت اگر راست ترین بیت صفر باشد، پس این ایندکس جایگاه راست ترین بیت 1 است و برعکس.

منطقی که پیاده سازی شد بر اساس شکل پایین است. به طور کلی در الگوریتم booth یک accumulator داریم که اساس محاسبه حاصل ضرب است. این accumulator هر بار مقداری شیفت میخورد و با حاصل قبلی جمع/تفریق می شود. این الگوریتم باید N بار اجرا شود که N تعداد بیت های اعداد ورودی است. در پیش گزارش منطق الگوریتم booth بیشتر توضیح داده شده است.

حال ما چه چیزی را با حاصل فعلی جمع یا تفریق میکنیم؟ شیفت خورده accumulator به اندازه shift_a در کد. این shift_a چیست؟ این مقدار شیفت فعلی و شیفت های کلاک های قبلی است. شیفت فعلی هم برابر با شماره ایندکس بیت راست مخالف راست ترین بیت است. این از منطق الگوریتم است که وقتی مثلاً 0111 در سمت راست multiplier داریم (که این البته در طول زمان کمی تغییر میکند و شیفت میخورد و multiplier است)، میتوان تا سه بار بدون مشکل شیفت

داد و در عین حال از منطق ترکیب 01 استفاده کرد. این در کد به نحو احسن پیاده سازی شد اما در ورژن قبلی دارای ایرداتی بود. بدین ترتیب خواسته سوال که پیاده سازی ویژگی شیفت چند بیتی است پیاده سازی می شود.



در مورد واحد کنترلی هم توضیح دهم که چند سیگنال داخلی و البته کنترل کننده مهم در این ماژول داریم. Done یعنی کی اجرای الگوریتم تمام می شود. مشخصا این به تعداد همان N است که قبلا هم توضیح داده شد. سیگنال های shift_a و shift_b داریم که تا حدی توضیح داده شد. Shift_b میزان شیفت در کلاک فعلی و shift_a میزان شیفت کل است، یعنی اینکه چقدر مقدار قبلی accumulator حالا شیفت می خورد. (یعنی acc از ابتدا تا حال چقدر شیفت می خورد).

سیگنال بعدی سیگنال start است. وقتی مدار در حال محاسبه پاسخ است، این سیگنال 1 می‌شود. این سیگنال بسیار مهمی است که در کد قبلی وجود نداشت. با اضافه کردن این سیگنال بسیاری از باگ های پیاده سازی حل شدند. سیگنال مهم دیگر سیگنال operation_select است. اینکه کدام عملیات (جمع یا تفریق) انجام شود. پس از شیفت عدد، اگر بیت سمت راست 1 باشد، یعنی به صورت 01 است و باید جمع صورت گیرد. اگر هم صفر باشد، یعنی 10 است و باید تفریق صورت گیرد. اینکه چه صورت گیرد در واحد کنترلی مشخص می‌شود و اینکه عملیات صورت گیرد، در datapath. سیگنال start هم نقش مهمی دارد. اگر $start = 0$ ، در این صورت در حالت reset هستیم و چون در ابتدا بیت سمت راست صفر است، باید جایگاه ایندکس راست ترین یک را به عنوان shift_b لحاظ کنیم. این هم موضوع مهمی است.

پس از پیاده سازی این مدار ها، این ماژول ها در ماژول booth_mult به هم متصل شدند که این ماژول نکته خاصی ندارد جز اینکه این بلوک در واقع interface کل مدار و top level entity است. برای تست واقعی مدار در آزمایشگاه باید ورودی ها و خروجی های این مدار در نظر گرفته شوند.

پس از این مدار ها هم ماژول تست پیاده سازی شد که به ازای تمام مقادیر ممکن N بیتی مدار را تست می‌کند. فقط کافی است در ابتدا N و متناسب با آن LOGN ست شوند.

توجه شود که خیلی تلاش شد تا این مدار یک مدار ضرب کننده با بیت های متغیر باشد. میتوان هر دو عدد N بیتی را با این مدار در هم ضرب کرد و صرفا محدود به اعداد با بیت ثابت (مثلا اعداد 4 بیتی) نیست.

این نتیجه تست مدار برای $N = 8$ است. یعنی برای 2^{16} عدد. خروجی نهایی تست حدود 5 6 ثانیه برای این تعداد بسیار عدد آماده می‌شود.

```
PS C:\Users\ASUS\OneDrive\Desktop\university\5\DSD_Lab\5\cicuit> vvp .\booth_mult.vvp
VCD info: dumpfile booth_mult.vcd opened for output.
----- Fortunately, all tests (65536) passed for N=8 -----
booth_mult_tb.v:63: $finish called at 4584985000 (1ps)
```

یا مثلا برای $N = 4$ نتیجه نهایی فوری و در کسری از ثانیه چاپ می‌شود.

(آپشن اینکه تعداد کل تست ها هم شمرده شود، اضافه شد تا به یقین ببینیم که مدار به درستی کار می‌کند.)

همچنین این مدار سنتز پذیر است و خدا را شکر بدون مشکل در کوارتوس سنتز شد. این نتیجه flow summary و RTL viewer است. توجه شود که برای RTL viewer می‌توان روی ریز بلوک های این بلوک کلی در کوارتوس کلیک کرد تا شماتیک آنها هم دیده شود. تمام عکس ها و فایل ها به پیوست ارسال می‌شود. همچنین این عکس ها در پایین هم قرار گرفته شدند:

Quartus II 64-Bit - C:/Users/ASUS/OneDrive/Desktop/university/5/DSD_Lab/5/circuit/booth_mult - booth_mult

File Edit View Project Assignments Processing Tools Window Help

Search altera.com

Project Navigator

Files

- control_unit.v
- datapath.v
- booth_mult.v
- find_first_different_bit.v
- booth_mult_tb.v

Table of Contents

- Flow Summary
- Flow Settings
- Flow Non-Default Global Settings
- Flow Elapsed Time
- Flow OS Summary
- Flow Log
- Analysis & Synthesis
- Fitter

Hierarchy Files Design Units IP Components

Tasks

Flow: Compilation Customize...

Task	Time
Compile Design	00:00:19
Analysis & Synthesis	00:00:04
Edit Settings	
View Report	
Analysis & Elaboration	
Partition Merge	
Netlist Viewers	
Design Assistant (Post-Mapping)	
I/O Assignment Analysis	
Early Timing Estimate	
Fitter (Place & Route)	00:00:09
Assembler (Generate programming files)	00:00:02
TimeQuest Timing Analysis	00:00:03
EDA Netlist Writer	00:00:01
Program Device (Open Programmer)	

Flow Summary

Flow Status: Successful - Wed Dec 03 18:36:31 2025

Quartus II 64-Bit Version: 13.1.0 Build 162 10/23/2013 SJ Web Edition

Revision Name: booth_mult

Top-level Entity Name: booth_mult

Family: Cyclone IV GX

Total logic elements: 161 / 14,400 (1 %)

Total combinational functions: 161 / 14,400 (1 %)

Dedicated logic registers: 29 / 14,400 (< 1 %)

Total registers: 29

Total pins: 35 / 81 (43 %)

Total virtual pins: 0

Total memory bits: 0 / 552,960 (0 %)

Embedded Multiplier 9-bit elements: 0

Total GXB Receiver Channel PCS: 0 / 2 (0 %)

Total GXB Receiver Channel PMA: 0 / 2 (0 %)

Total GXB Transmitter Channel PCS: 0 / 2 (0 %)

Total GXB Transmitter Channel PMA: 0 / 2 (0 %)

Total PLLs: 0 / 3 (0 %)

Device: EP4CGX15BF14C6

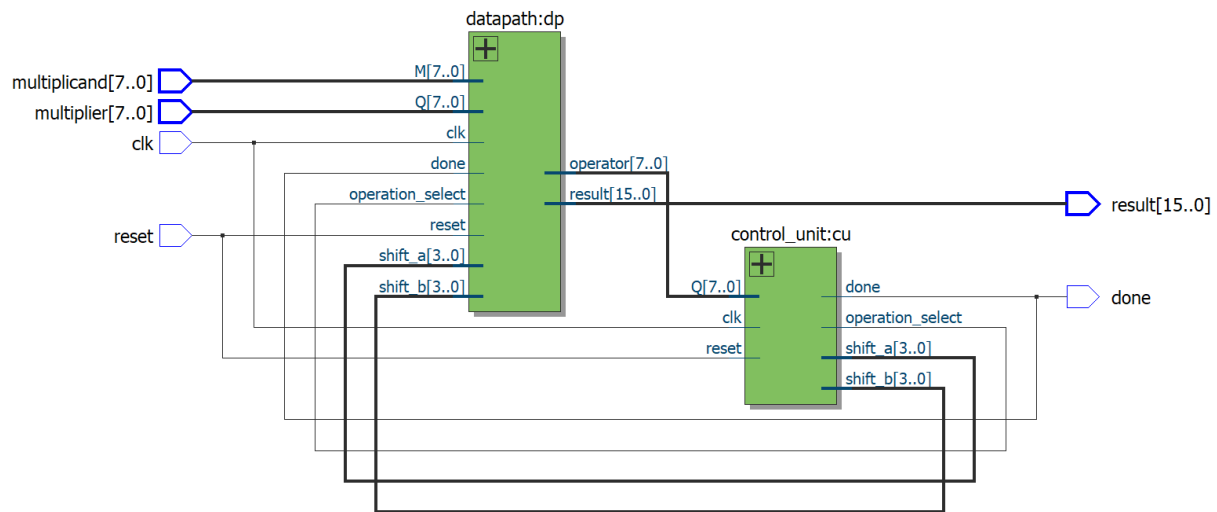
Timing Models: Final

Messages

Type ID Flag Message

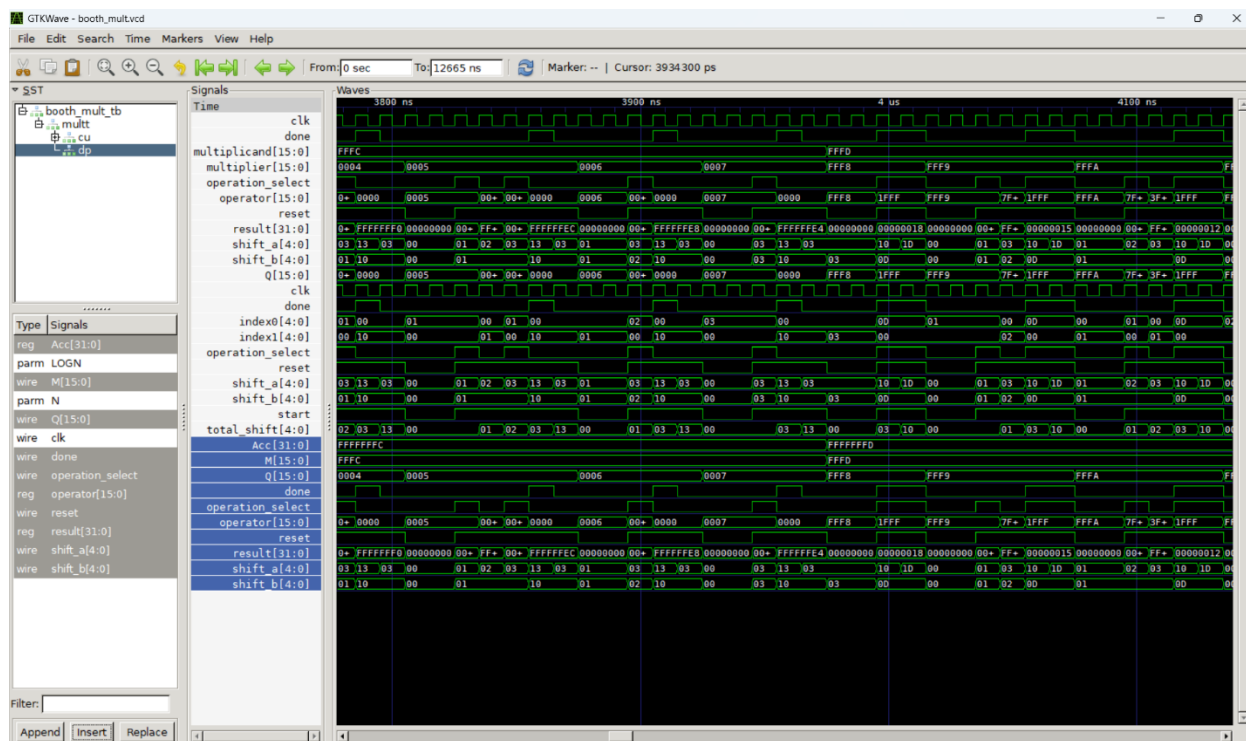
- 204019 Generated file booth_mult_min_1200mv_0c_v_fast.sdo in folder "C:/Users/ASUS/OneDrive/Desktop/university/5/DSD_Lab/5/circuit/simulation/modelsim/" for EDA simulation tool
- 204019 Generated file booth_mult_v.sdo in folder "C:/Users/ASUS/OneDrive/Desktop/university/5/DSD_Lab/5/circuit/simulation/modelsim/" for EDA simulation tool
- 293000 Quartus II 64-Bit EDA Netlist Writer was successful. 0 errors, 0 warnings
- 293000 Quartus II Full Compilation was successful. 0 errors, 27 warnings

System Processing (155) / 100% 00:00:19



دو شکل پایین هم شماتیک مدار datapath و control_unit در RTL-viewer است:

به کمک GTK هم waveform مدار دیده میشود که بسیار بسیار بزرگ است و بنده به طور رندوم از سه جای آن عکس گرفته و به پیوست ارسال کردم. همچنین یک عکس هم در پایین قرار میدهم:



این گزارش تکمیلی و اصلاحی برای پیاده سازی مدار بود که در تاریخ 1404/9/12 نوشته شد.